

Project Hail Mary: The Adrian Descent

Architecting a Physics-Based RL Environment

Introduction: The Mission

Ryland Grace and his Eridian lab partner, Rocky, have a massive problem. Orbiting the planet Adrian in the Tau Ceti system, they need to send an automated probe deep into its turbulent, pitch-black atmosphere to retrieve a sample of Astrophage-eating Taumoeba.

Because of the unpredictable Adrian storms and the communication delay from the *Hail Mary*, hardcoded instructions will fail. "Amaze! You write smart-code. Probe lands safe, question?" Rocky taps excitedly on his xenonite barrier.

Your objective is to train a Tabular Q-Learning agent to autonomously control the probe's spin-drive engines, executing a soft atmospheric hover-catch despite the massive gravity and chaotic winds of Adrian. You will architect the physics environment, formulate the Markov Decision Process (MDP), design the Q-table, and write the execution loops from scratch.

Architecture Choice: You may either cleanly separate your codebase into multiple files (`env.py`, `agent.py`, `train.py`, `run.py`) or encapsulate the entire mission in a single Jupyter Notebook (`adrian_descent.ipynb`).

Part 1: The Deterministic Baseline (Kinematics)

Before introducing AI, you must build the mathematical reality of Adrian. Inside `env.py` (or an Environment cell in your notebook), create a `ProbeEnv` class. Your environment tracks two continuous state variables: **Altitude** (h) and **Velocity** (v).

Constants of Adrian

Use the following lore-accurate physics constants:

- Probe Mass (m):** 1000.0 kg
- Adrian Surface Gravity (g):** 13.7 m/s^2 ($\sim 1.4G$)
- Radius of Adrian (R_{adrian}):** 10,700,000 m ($1.68R_{\oplus}$)
- Atmospheric Composition:** 91% Carbon Dioxide, 7% Methane, 1% Argon
- Atmospheric Drag Coefficient (k_{drag}):** 2.0 (High density)
- Spin-Drive Thrust (F_{thrust}):** 25,000 N
- Integration Time Step (dt):** 0.1 s

Newtonian Kinematics & Euler Integration

At every time step ($dt = 0.1s$), you must calculate the net force acting on the probe:

$$F_{net} = F_{gravity} + F_{thrust} + F_{drag}$$

1. **Gravity:** Adrian's gravity increases slightly as the probe descends.

$$F_{gravity} = -m \cdot g \cdot \left(1 - \frac{h}{R_{adrian}}\right)$$

2. **Thrust:** The agent has a binary action space: 0 (Spin-Drive OFF) or 1 (Spin-Drive ON). When ON, F_{thrust} is applied.

3. **Drag:** The thick atmosphere opposes the direction of movement.

$$F_{drag} = k_{drag} \cdot v^2 \cdot \text{sign}(-v)$$

4. **Numerical Integration:** We convert force to acceleration ($a = F_{net}/m$) and update the state using *Euler Integration*:

$$v_{t+1} = v_t + a \cdot dt \quad \implies \quad h_{t+1} = h_t + v_{t+1} \cdot dt$$

Rocky's Analytics: Terminal Velocity

"Math is math! Calculate speed, yes?"

Before you write any code, calculate the terminal velocity of the probe if the spin-drive completely fails (i.e., when $F_{gravity}$ perfectly balances F_{drag}). Is this velocity within the safe target-catch limit? What does this tell you about the difficulty of the control problem?

Rocky's Analytics: Maximum Net Force

With the spin-drive engaged ($a = 1$) and the probe perfectly stationary ($v = 0$), calculate the net acceleration. Is the thrust strong enough to make the probe ascend against Adrian's gravity? What does this imply for an agent that wants to "avoid" completing the mission?

Your Task: Implement the following core methods for `ProbeEnv`:

- `reset()`: Resets the probe to the drop altitude (e.g., 1000m), zero velocity, and zero steps. It must return the initial state tuple.
- `step(action)`: The core physics loop. It receives an action (0 or 1), applies the correct forces, runs Euler integration to update altitude and velocity, checks for terminal conditions (like hitting the ground), and returns the standard RL tuple: `(next_state, reward, done)`.

Part 2: The Stochastic Environment (MDP Formulation)

Adrian's storms are legendary. A naive agent that memorizes a perfectly timed sequence of engine burns will be destroyed the moment an unexpected gust hits it. You must convert your deterministic physics engine into a stochastic **Markov Decision Process (MDP)**.

The Formal Markov Property

The *Markov Property* states that future dynamics depend strictly on the present state and action, completely ignoring the history:

$$\mathbb{P}[S_{t+1}|S_t, A_t] = \mathbb{P}[S_{t+1}|S_1, A_1, \dots, S_t, A_t]$$

As long as your state tuple encapsulates all relevant physics parameters, the probe can make an optimal survival decision from a single snapshot in time.

Design Question: Memory in the Storm

Real wind does not change entirely randomly every second; it has *momentum*. How does a Transition Probability Matrix (where the next state depends heavily on the current state) accurately simulate weather patterns better than simply rolling a random 3-sided die at every time step?

Your Task: Introduce a "Wind" variable into your environment state.

- Define three wind states: 0: Calm (1.0x drag), 1: Gusty (1.5x drag), 2: Adrian Gale (2.5x drag).
- Create a **Transition Probability Matrix** (3×3). Modify `step()` to randomly transition the wind state based on these probabilities.
- Multiply your F_{drag} by this new wind multiplier. Your raw state emitted by `step()` and `reset()` is now: (h, v, wind_idx).

Part 3: Programming the Agent (Q-Learning)

In `agent.py` (or your Agent cell), create `ProbeAgent`. Altitude and velocity are continuous floats, meaning an infinite number of states exist.

The Curse of Dimensionality

To use a tabular Q-learning approach, you must discretize continuous variables into buckets. However, state spaces grow *exponentially* with each new dimension.

If you use 50 buckets for altitude, 50 for velocity, and 3 wind states, your Q-table requires $50 \times 50 \times 3 \times 2 = 15,000$ entries. If you wanted "more precision" and used 500 buckets each, the table explodes to **1.5 Million** entries. Most of these states will never be visited, making learning impossible in a reasonable timeframe. You must strike a balance between resolution and learnability.

Design Question: Exploration vs. Exploitation

Early in training, the agent knows nothing and must explore randomly. Over time, it must use its acquired knowledge to survive. We manage this using an ϵ -greedy policy. If you start with $\epsilon = 1.0$ and decay it by multiplying it by 0.999 every episode, what will the exploration rate be after 5,000 episodes? Is the agent mostly exploring or mostly exploiting by that point?

Your Task: Implement the following methods for `ProbeAgent`:

- `discretize_state(raw_state)`: Takes the continuous (`h`, `v`, `wind`) floats generated by the environment and maps them to discrete integer indices (e.g., using `np.digitize`) so they can be used to query the Q-table arrays.
- `choose_action(state)`: Implements the ϵ -greedy policy. Rolls a random probability to either return a random action (explore) or the mathematical maximum action from the Q-table (exploit).
- `learn(state, action, reward, next_state, done)`: Evaluates the Bellman equation and adds the Temporal Difference (TD) error to update the specific Q-table entry for the chosen state and action.

Part 4: Astrophage Efficiency & The "Cowardly Agent"

Designing the reward signal is the hardest part. If your rewards are not strictly balanced, your agent will exploit loopholes—a phenomenon known as the *Cowardly Agent*.

Design Question: Pitfalls of the Cowardly Agent

If you heavily penalize the probe for crashing, but only apply a tiny penalty for using Astrophage fuel, what degenerate behaviour might the agent learn to do instead of landing? Conversely, if you only give a massive reward for a successful catch (+1000) but no intermediate rewards during the descent, why might the agent struggle to learn anything at all?

Your Task: Shape a robust reward structure. You must define the exact mathematical values for the following events:

- **Engine Burn:** A step-by-step fuel cost when Thrust is ON.
- **Soft Target Catch:** A massive terminal reward for reaching $h \leq 0$ with a safe velocity (e.g., $v \geq -3.0$).
- **Crushed by Pressure:** A terminal penalty for reaching $h \leq 0$ too fast (Hint: scale this penalty dynamically based on the impact velocity).
- **Runaway Probe:** A penalty and force-termination condition if the agent tries to fly back to orbit ($h > 1200m$) or hovers indefinitely (exceeding a `max_steps` limit).

Note: There is no single correct answer. You must experiment, watch the probe fail, and tune your numbers until the agent is forced to land.

Part 5: Mission Control (Training & Evaluation)

Writing the environment and agent is only half the battle. You must now train the probe and mathematically prove that it has learned a safe descent policy.

Your Task: Architect the execution loops in `train.py` and `run.py` (or your final notebook cells):

- **Training Loop:** Instantiate the environment and agent. Run a training loop for at least 15,000 to 20,000 episodes. You must track the total reward and the success/crash result of every episode. At the end of training, use `matplotlib` to plot two graphs (ideally as subplots): a moving average of the Episode Reward, and a moving average of the Landing Success Rate (%).
- **Evaluation:** Run a *single* purely greedy test episode ($\epsilon = 0.0$). Call the provided `render_probe_ascii()` function at every step so you can watch your probe descend in the terminal (or notebook output).

Design Question: The Asymptote of Reward

When plotting your episode reward curve, you will notice that the moving average eventually flattens out, but it never actually touches the theoretical maximum score (e.g., exactly +1000). Conceptually, why is it mathematically impossible for the agent to achieve a perfect, unpenalized score?

Part 6: Grace's Lesson (Bridge to the OS Scheduler)

Ryland Grace looks at your successful simulation and taps the monitor. *"You know,"* he says, *"what you just built here is exactly how an Operating System kernel should work."*

Look closely at the architecture. The probe tracks volatile variables (Altitude, Velocity), operates under stochastic conditions (Adrian's Wind), and requires the agent to make a high-stakes decision to optimize a delayed reward while minimizing a continuous penalty (Astrophage fuel).

An OS kernel CPU scheduler operates on the exact same mathematical philosophy:

- **The State:** Instead of Altitude and Velocity, the kernel tracks Queue Length and Average Burst Time.
- **The Noise:** Instead of Adrian's storms, the kernel deals with stochastic, unpredictable user inputs and network packet arrivals.
- **The Action:** Instead of Thrust ON/OFF, the scheduler picks between algorithms like Round Robin (RR) or Shortest Job First (SJF).
- **The Reward:** Instead of avoiding being crushed by gravity, the scheduler avoids high P90 latency spikes while optimizing throughput.

Master the Q-Table mechanics here, and you are ready to compile your own RL-driven OS Kernel.

Deliverables

Submit a `.zip` archive (if using separate files) or a single `.ipynb` file containing the following:

1. Your `ProbeEnv` and `ProbeAgent` classes.
2. Your training and execution loops.
3. `learning_curve.png` (or displayed directly in the notebook output).
4. `report.md` or `.pdf` (Containing brief answers to the **6 Design Questions**. If using a Notebook, these can be written directly in Markdown cells).

Amaze! Bonus Challenge: Variable Spin-Drive

Right now, the probe's spin-drive has a binary action space: OFF (0%) or ON (100%). Real spacecraft throttle their engines to maintain precise descent rates and optimize fuel efficiency.

The Challenge: Extend your action space to include a partial-thrust level (e.g., 0: OFF, 1: 50% Thrust, 2: 100% Thrust). Retrain your agent.

- Does a finer-grained action space help the agent achieve a softer landing?
- How does this additional action mathematically change the size of your Q-table, and does it increase the training time required to converge?

Provided Tooling: The Terminal Renderer

To visualize your trained agent in real-time, copy this high-framerate ASCII renderer into your testing script or cell. Call it inside your testing loop immediately *after* the agent selects an action, but *before* you step the environment.

```

1 import os
2 import time
3 from IPython.display import clear_output # For Jupyter Notebooks
4
5 def render_probe_ascii(h, max_h, v, action, wind, step_count, is_jupyter=False)
6 :
7     """A high-framerate, pure ASCII terminal renderer."""
8
9     if is_jupyter:
10         clear_output(wait=True)
11     else:
12         os.system('clear' if os.name == 'posix' else 'cls')
13
14     term_lines = 40
15
16     if h > 150.0:
17         display_max = max_h
18         zoom_str = "[ CAMERA: WIDE ANGLE (1000m) ]"
19     else:
20         display_max = 150.0
21         zoom_str = "[ CAMERA: TARGET APPROACH (150m) ]"
22
23     pos = int((h / display_max) * term_lines)
24     pos = max(0, min(term_lines, pos))
25
26     wind_strs = ["~ Calm ~", "Gusty", "Adrian Gale"]
27     thrust_str = "[####] ON " if action == 1 else "[    ] OFF"
28
29     print(f" T+{step_count:03d}s | ALT: {h:6.1f}m | VEL: {v:7.1f}m/s | THRUST:
30 {thrust_str} | WIND: {wind_strs[wind]}")
31     print(f" {zoom_str}")
32     print("-" * 75)
33
34     for i in range(term_lines, -1, -1):
35         if i == pos:
36             if action == 1:
37                 print("                /\\"")
38                 print("                |  |")
39                 print("                /WW\\")
40                 print("                ||  <-- spin-drive")
41             else:
42                 print("                /\\"")
43                 print("                |  |")
44                 print("                /--\\")
45                 print("                ")
46         else:
47             if i % 10 == 0:
48                 print(f" {int((i/term_lines)*display_max):4d}m
49 +-----")
50             else:
51                 print("          |")
52
53     print("===== [ TAUMOEBA TARGET ] =====")
54 )
55
56 time.sleep(0.04)

```